Search Algorithms: Warehouse Delivery Robot

Intelligent Systems - Task 1

Team Members	ID
عمر محمد عبد الحميد السيد متولي	2403249589
عبدالله محمد مروان الناجي	2403246317
خيري عماد خيري	2403244009

GitHub link: https://github.com/abdallah-ai04/warehouse-robot-task1

1. Problem Formulation

A robot must move from S to G on a grid, one cell at a time (Up, Down, Left, Right), and cannot enter walls. Entering a normal floor cell costs 1, a rough floor cell costs 3, and entering G costs 1 (S is not counted).

Element	Definition
State	The robot's cell (row, column)
Initial state	Cell of S, found by scanning the grid
Actions	Up, Down, Left, Right (inside the grid and not into #)
Transition cost	1 for '.' and 'G', 3 for '~'
Goal test	state == cell of G
Solution quality	Minimum total path cost (not minimum number of moves)

2. Algorithms

Uniform Cost Search (UCS) - uninformed. Keeps a priority queue ordered by g(n), the cost from S to n. It always expands the cheapest node first, so the first time G is expanded the path is optimal. A closed set and a best-g table avoid re-expanding cells.

A* Search - informed. Same as UCS but the priority is f(n) = g(n) + h(n).

Heuristic. h(n) = Manhattan distance to G = $|r - r_G| + |c - c_G|$. It is admissible because the robot moves in 4 directions and every move costs at least 1, so the real cost can never be smaller than the distance. It is also consistent, so A* is optimal and never needs to reopen closed nodes.

Implementation notes. Written in pure Python with heapq only (no path-finding libraries). Both algorithms share one function; UCS uses h = 0. Metrics recorded: path length, path cost, nodes expanded, maximum frontier size and run time.

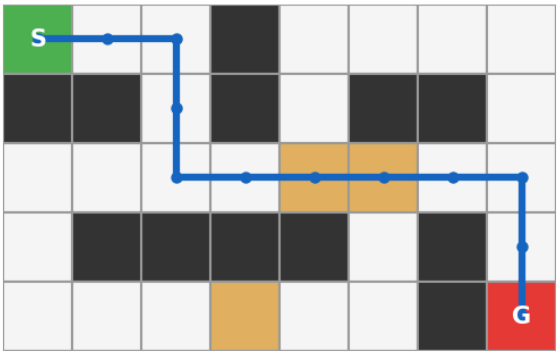
3. Results

Map	Algorithm	Length	Cost	Expanded	Max frontier	Time (ms)
MAP_A	UCS	11	15	28	4	0.076
MAP_A	A*	11	15	21	5	0.072
MAP_B	UCS	12	12	31	4	0.083
MAP_B	A*	12	12	15	7	0.050
MAP_C	UCS	-	-	13	3	0.038
MAP_C	A*	-	-	13	3	0.060

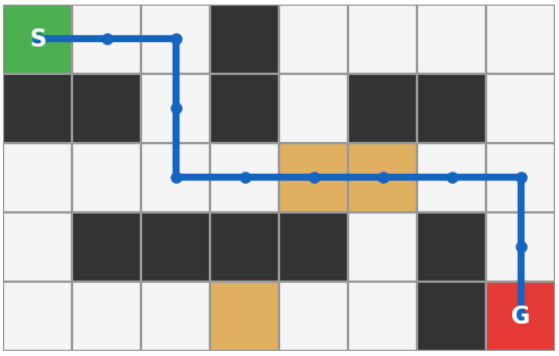
Length = number of moves. Run times are tiny and vary slightly between runs. MAP_C has no solution, so both algorithms exhaust all reachable cells (13) and print "No path found".

4. Path Screenshots

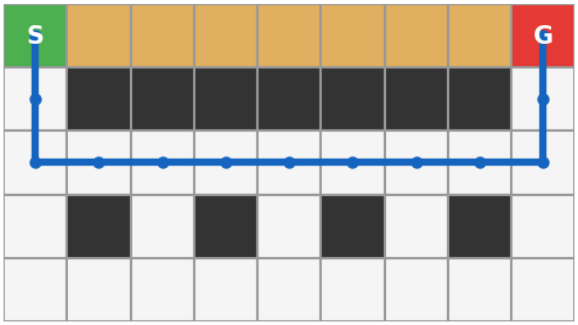
MAP_A - UCS



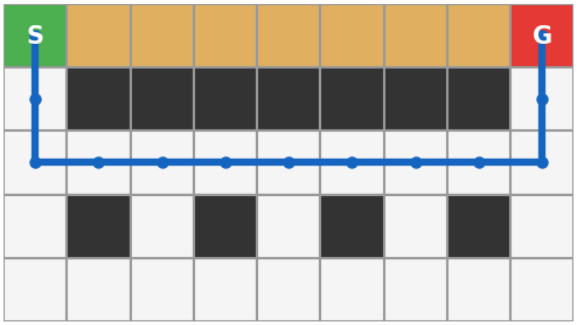
MAP_A - A*



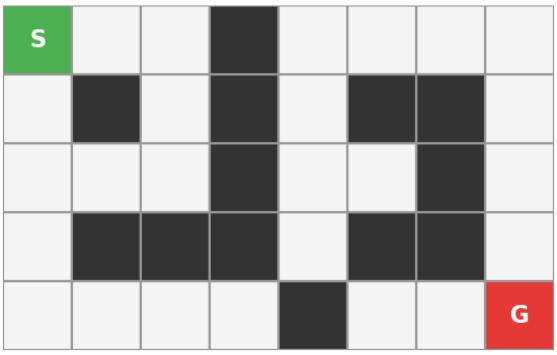
MAP_B - UCS



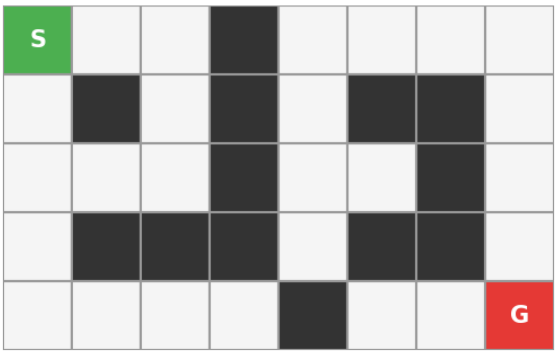
MAP_B - A*



MAP_C - UCS (No path found)



MAP_C - A* (No path found)



Console output with * marking the path (MAP_B):

```
UCS / A* on MAP_B:
S~~~~~~~G
*#######*
*********
.#.#.#.#.
.........
```

MAP_B is the trap: the direct top route is only 8 moves but costs 22 (seven rough cells), while the detour through the bottom corridor is 12 moves but costs only 12. Both algorithms choose the cheaper one.

5. Comparison

Criterion	UCS	A*
Cheaper path?	Optimal (A: 15, B: 12)	Optimal, same cost (A: 15, B: 12)
Nodes expanded	A: 28, B: 31, C: 13	A: 21, B: 15, C: 13
Max frontier	Smaller (4)	Slightly larger (5 to 7)

Needs heuristic?	No	Yes (Manhattan)

Both algorithms found paths of the same (minimum) cost, because both are optimal here. A* expanded fewer nodes on MAP_A (21 vs 28) and MAP_B (15 vs 31) since the heuristic steers the search toward G, at the price of a slightly larger frontier. On MAP_C both explored the same 13 cells, because when no path exists every reachable cell must be checked.

6. Conclusion

For this robot I would choose **A***: it guarantees the cheapest path (the heuristic is admissible and consistent) while expanding fewer nodes than UCS, and its extra memory is negligible. UCS is a good fallback when no heuristic is available. BFS or DFS would not be suitable since they ignore floor costs and can return expensive paths (as the MAP_B trap shows).

7. How to Run

python warehouse_robot.py (uses GRID at the top of the file; change GRID = MAP_A to MAP_B or MAP_C). python warehouse_robot.py --all runs all maps and prints the results table.